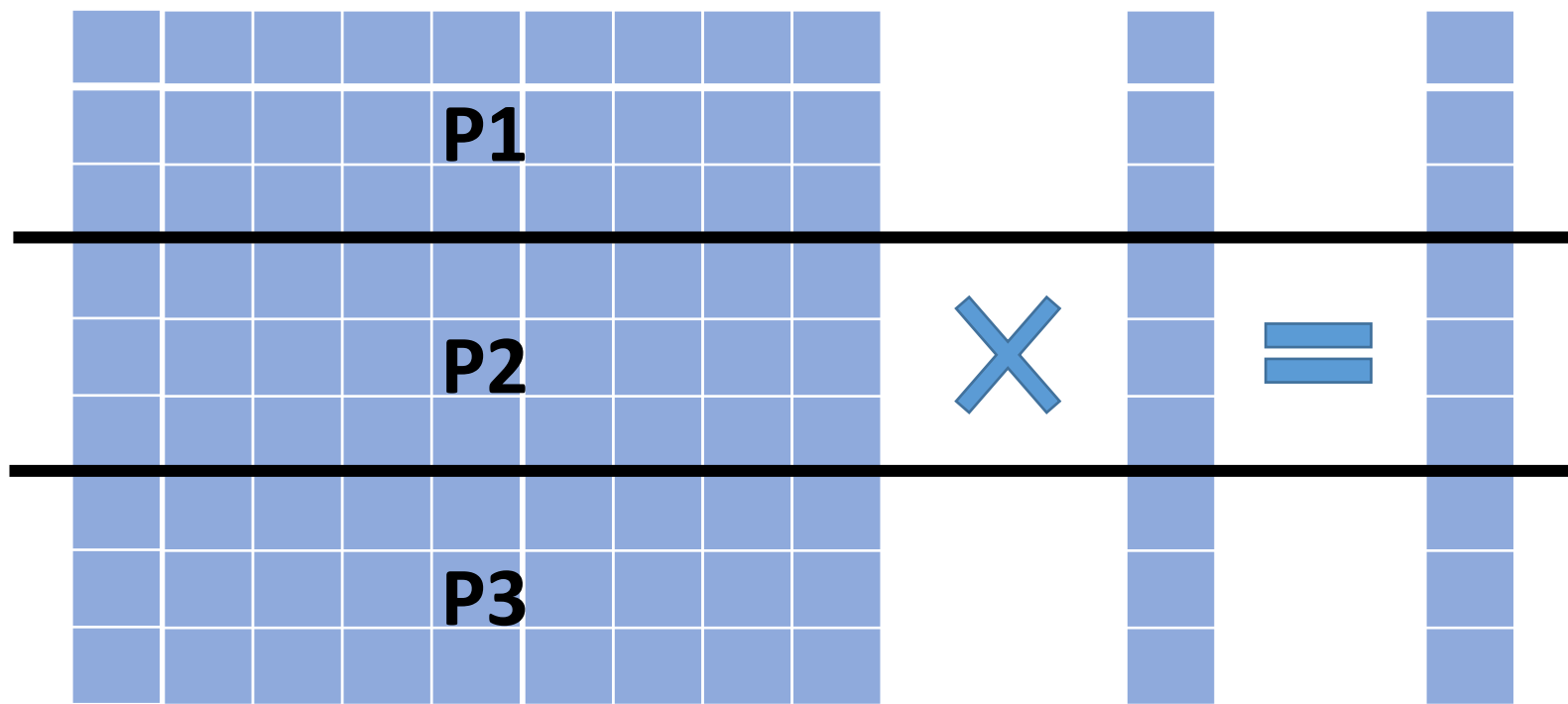


Non-blocking Collectives

Lecture 20

March 30, 2026

Matrix Vector Multiplication – Recap



P = 3

Decomposition

Assignment

Orchestration

- MPI_Allgather/
MPI_Bcast
- MPI_Scatter
- MPI_Gather

- Initial communication
 - Distribute (read by process 0) or parallel reads
- Final communication

Write the Code

- Obtaining the vector (length N)
 - MPI_Bcast (in case the entire vector was read by 0)
 - MPI_Allgather (in case of parallel reads)
- Obtaining the matrix (in case the matrix was read by 0)
 - MPI_Scatter
- Computation: $\text{subMatrix}[R][C] \times \text{vector}[R]$
- Final communication
 - MPI_Gather

Non-blocking Collectives

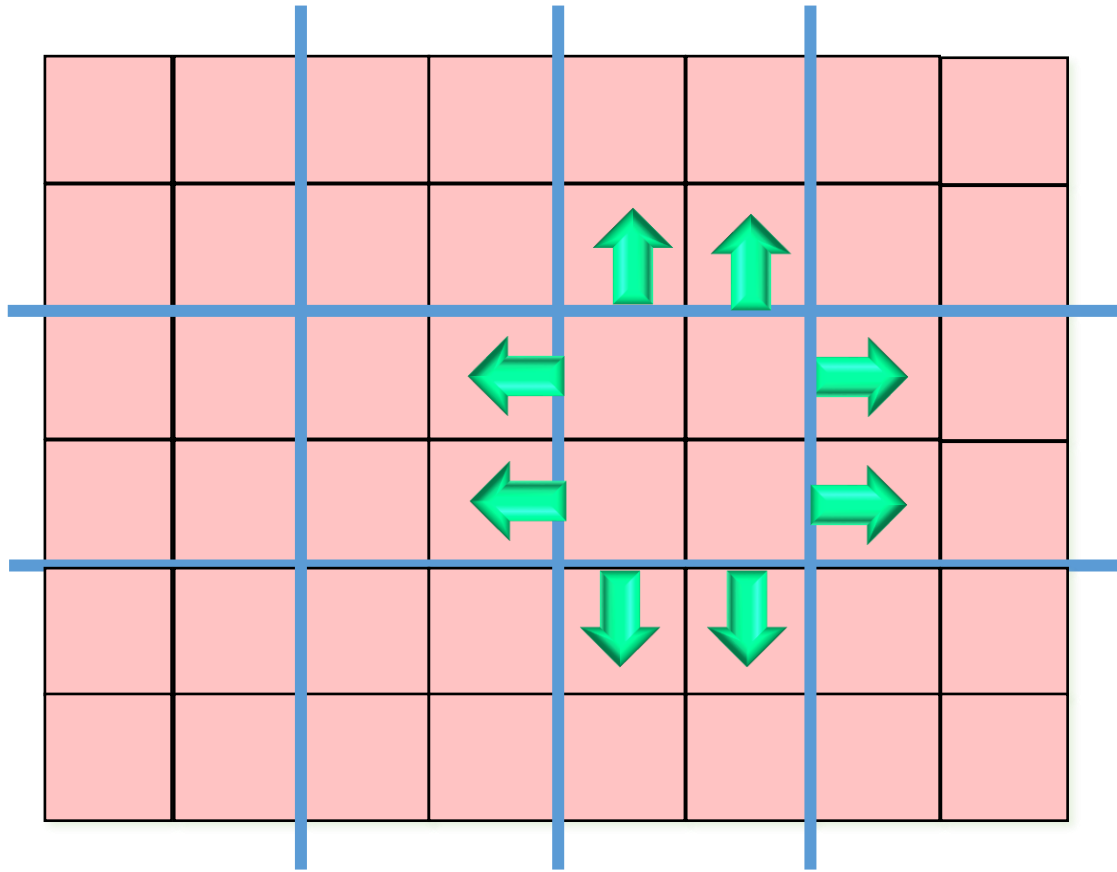
```
int MPI_Ibcast (  
    void *buffer,  
    int count,  
    MPI_Datatype datatype,  
    int root,  
    MPI_Comm comm,  
    MPI_Request *request  
)
```

```
int MPI_Igather (  
    const void *sendbuf,  
    int sendcount,  
    MPI_Datatype sendtype,  
    void *recvbuf,  
    int recvcount,  
    MPI_Datatype recvtype,  
    int root,  
    MPI_Comm comm,  
    MPI_Request *request  
)
```

Code Timing

```
double sTime = MPI_Wtime();  
for (int i=0; i<ROUNDS; i++)  
    MPI_Ibcast(buf, count, MPI_INT, 1, MPI_COMM_WORLD,  
    &request[i]);  
MPI_Waitall (ROUNDS,request,status);  
double eTime = MPI_Wtime();
```

2D Domain Decomposition – Halo Exchange



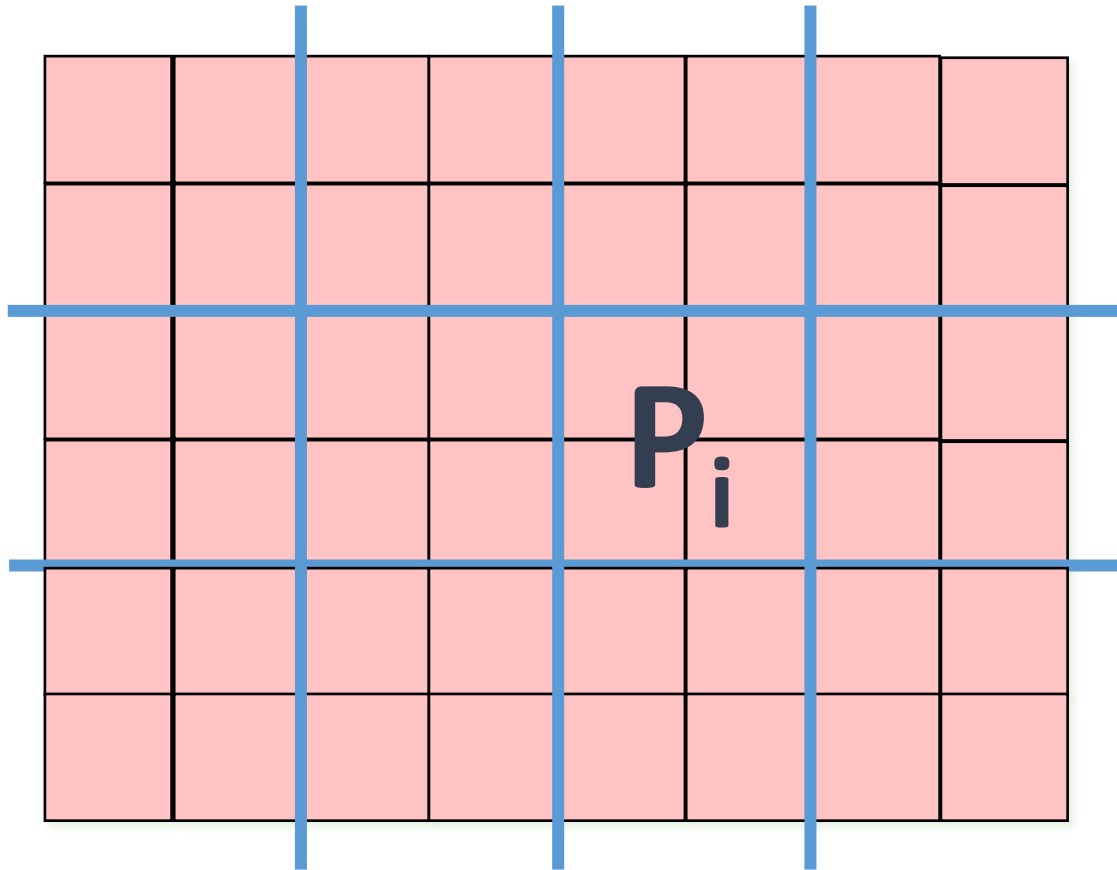
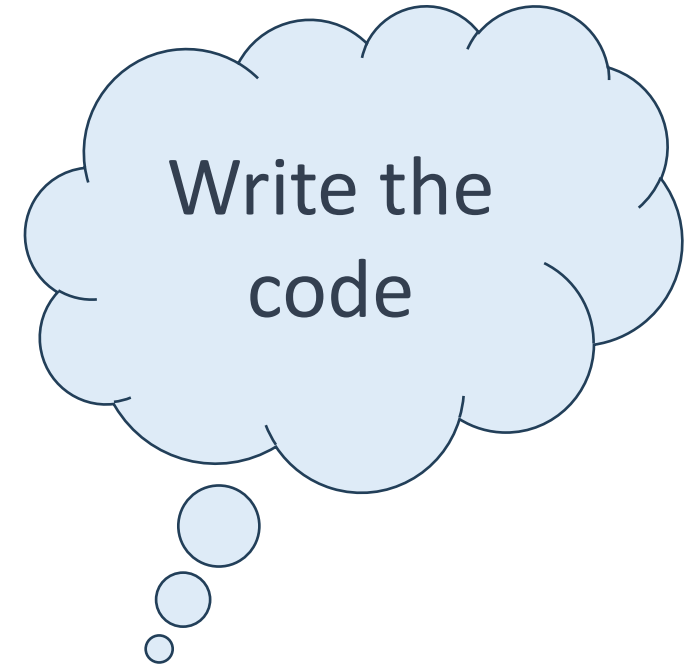
`float data[N][N]; // owned by a process`

Assume that the following are used for halo exchange communications

- `MPI_Gather` (for receiving from neighbours)
- `MPI_Scatter` (to send to neighbours)

Write the code

Create Sub-communicators



color = ?

How many MPI_Comm_split?

How many sub-communicators
should a rank belong to?

Example Code

```
// create the required number of comm[i] per rank -- Homework
for (i=0; i<4; i++)    // for the four neighbours that rank i will receive from
    MPI_Scatter(sendbuf[i], ct, dt, recvbuf[i], ct, dt, 0, comm[i]);
```

Will this work?

```
for (i=0; i<4; i++)
    MPI_Isscatter(sendbuf[i], ct, dt, recvbuf[i], ct, dt, 0, comm[i], &request[i]);
MPI_Waitall
```


Timings for MPI_Scatter vs. MPI_Iscatter

args: <Data size> <ROUNDS>

```
mpirun -np 4 -hosts csews100:2,csews200:2,csews199:2,csews198:2 ./exe 200000  
50
```

6.126909	0.815354	MPI_Scatter
----------	----------	-------------

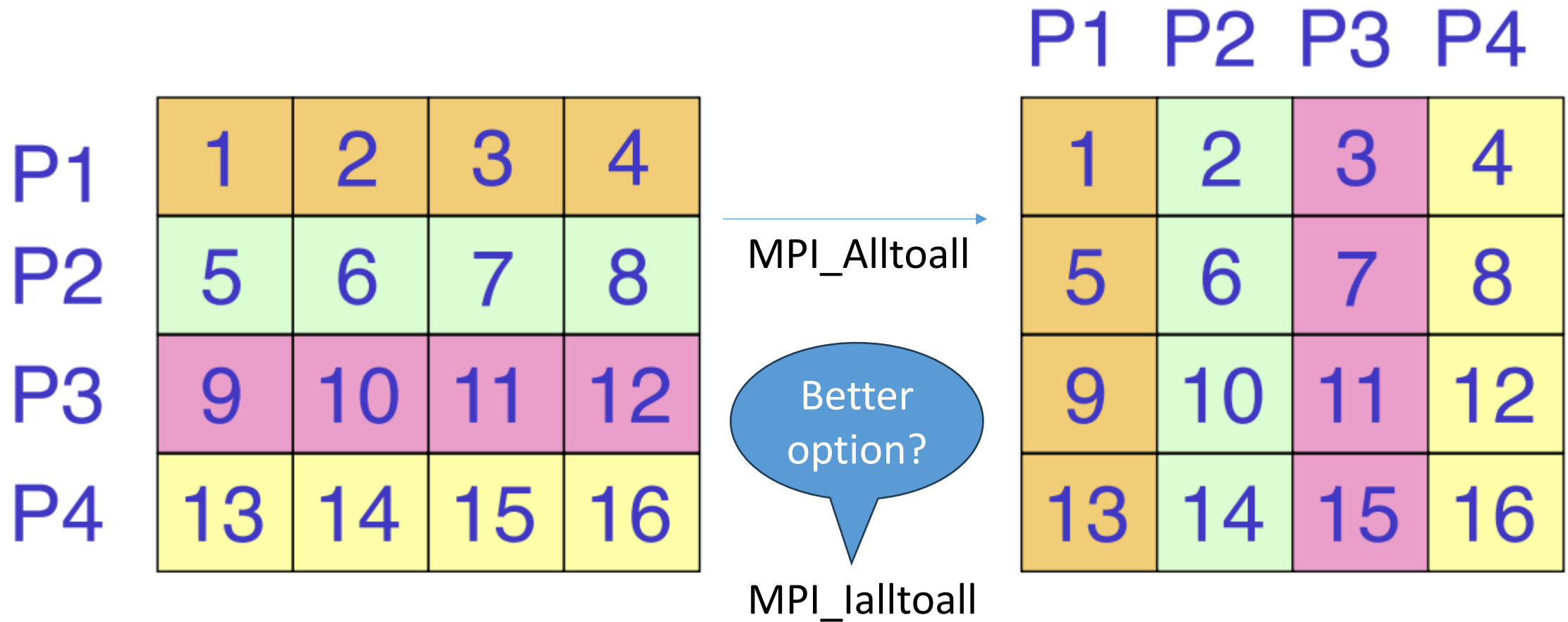
3.899728	0.815354	MPI_Iscatter
----------	----------	--------------

```
mpirun -np 16 -hosts csews180:4,csews200:4,csews197:4,csews198:4 ./exe 200000  
50
```

12.003180	0.378920	MPI_Scatter
-----------	----------	-------------

10.906063	0.378920	MPI_Iscatter
-----------	----------	--------------

Matrix Transpose



Credit: ICTP

Parallel I/O

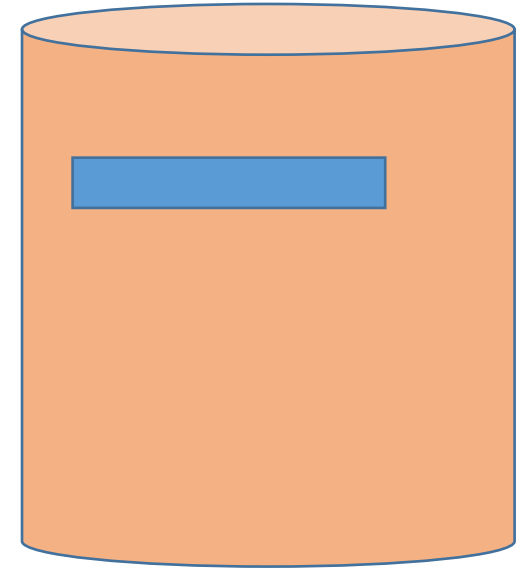
Sequential File Handling

```
char mystring[] = "Hello world"
```

```
FILE *fp = fopen ("/data/smallfile", "w")
```

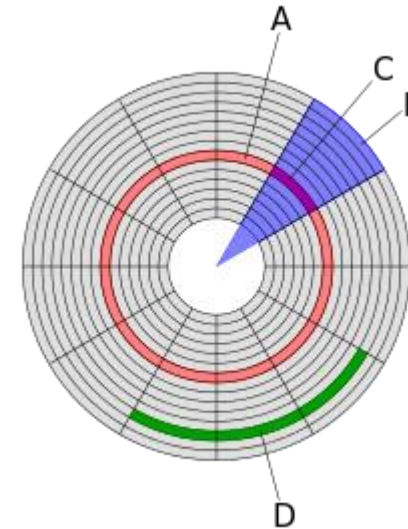
```
fwrite (mystring, sizeof(char), sizeof(mystring), fp)
```

```
fclose (fp)
```



Hard Disk Drive

- One process reads/writes to a file
- Files are stored on hard disk drives
 - Rotating disks
 - Read/write heads
 - Sequential access
 - Seek time + Rotational latency



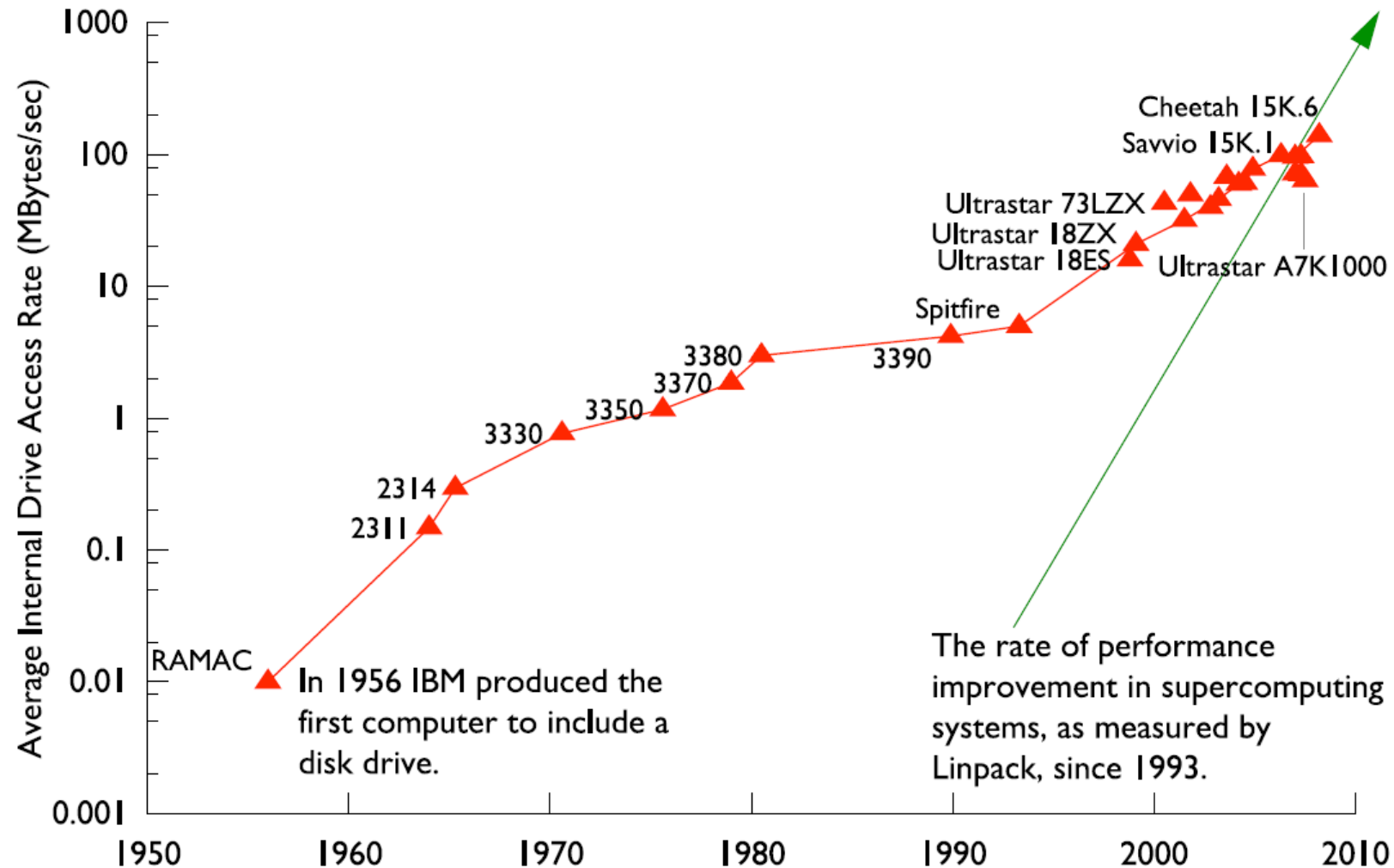
- A. Track
- B. Geometrical sector
- C. Track sector
- D. Cluster

[Source: Wikipedia]

- Mechanical device
- Magnetic storage medium
- Primary persistent storage device

<https://www.youtube.com/watch?v=sG2sGd5XxM4>

Disk Access Rates

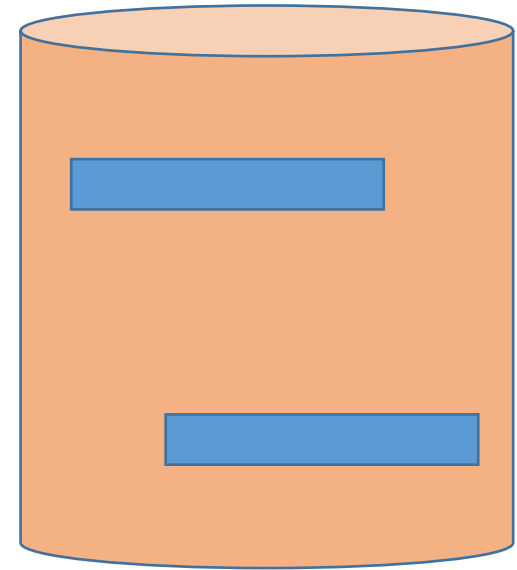


[Credit: R. Ross, ANL and R. Freitas, IBM]

Storage Devices

Access speed (write bandwidth)

- HDD
 - SSD
 - NVRAM
 - RAM
- ~ 300 MB/s
 - ~ 800 MB/s
 - ~ 1 - 2 GB/s
 - ~ 1 GB/s



I/O Bandwidths

```
pmalakar@csews5:~$ dd of=/dev/zero if=testfile bs=1K count=1000K
1024000+0 records in
1024000+0 records out
1048576000 bytes (1.0 GB, 1000 MiB) copied, 1.42756 s, 735 MB/s
pmalakar@csews5:~$
pmalakar@csews5:~$
pmalakar@csews5:~$ dd of=/dev/zero if=testfile bs=1K count=1000K
1024000+0 records in
1024000+0 records out
1048576000 bytes (1.0 GB, 1000 MiB) copied, 1.29833 s, 808 MB/s
pmalakar@csews5:~$
```

Read bandwidth

```
1024000+0 records in
1024000+0 records out
1048576000 bytes (1.0 GB, 1000 MiB) copied, 12.4325 s, 84.3 MB/s
pmalakar@csews5:~$
pmalakar@csews5:~$ dd if=/dev/zero of=/tmp/testfile bs=1K count=1000K
1024000+0 records in
1024000+0 records out
1048576000 bytes (1.0 GB, 1000 MiB) copied, 2.42031 s, 433 MB/s
pmalakar@csews5:~$ dd if=/dev/zero of=/tmp/testfile bs=1K count=1000K
1024000+0 records in
1024000+0 records out
1048576000 bytes (1.0 GB, 1000 MiB) copied, 7.34824 s, 143 MB/s
```

Write bandwidth

Read/Write Bandwidths (Today's run)

Read bandwidth

```
CS633 $ dd of=/dev/zero if=/tmp/wtestfile bs=1K count=1000K
1024000+0 records in
1024000+0 records out
1048576000 bytes (1.0 GB, 1000 MiB) copied, 0.319915 s, 3.3 GB/s
CS633 $ dd if=/dev/zero of=/tmp/wtestfile bs=1K count=1000K
1024000+0 records in
1024000+0 records out
1048576000 bytes (1.0 GB, 1000 MiB) copied, 0.785825 s, 1.3 GB/s
CS633 $ █
```

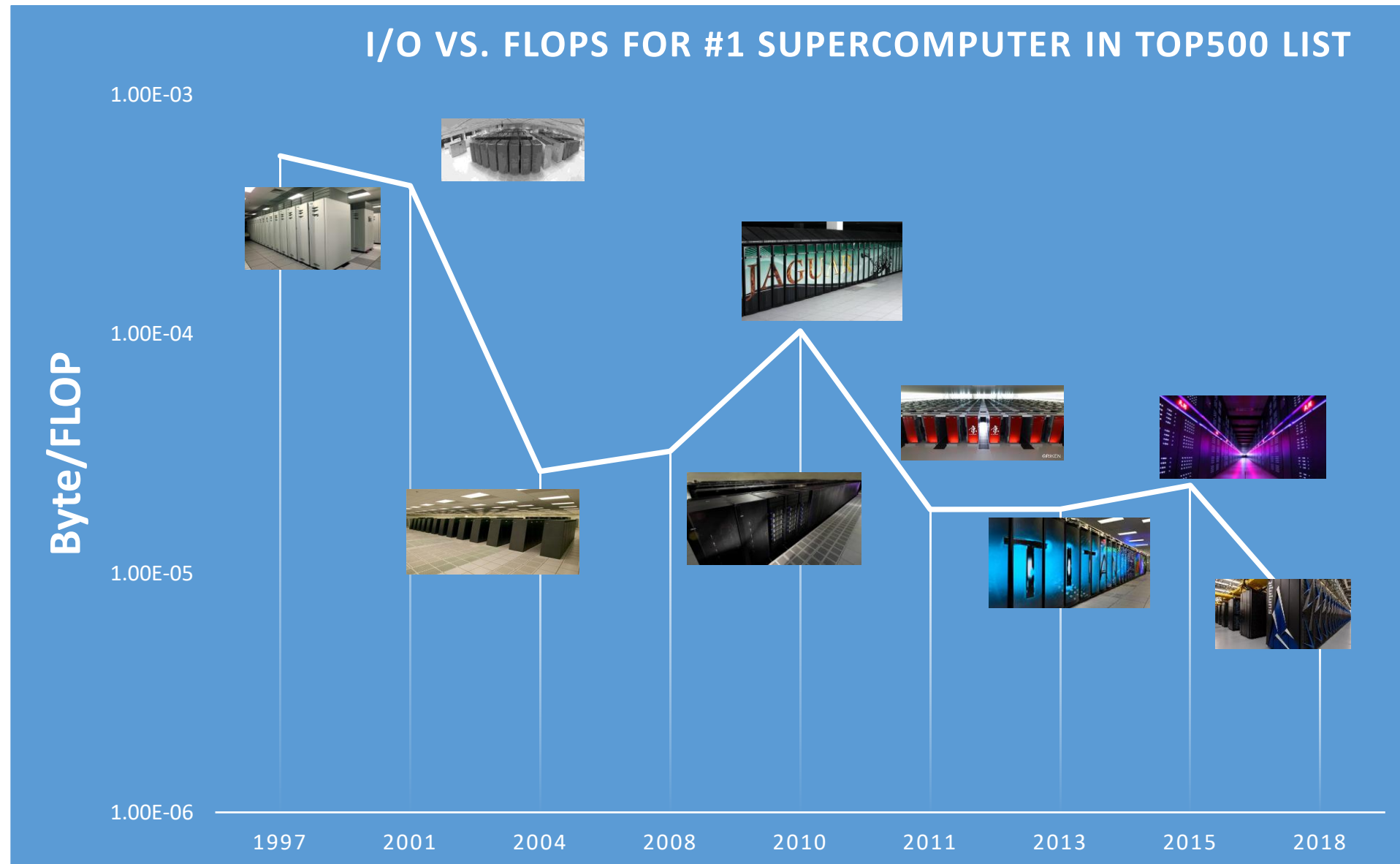
Write bandwidth

Data Requirements

Application	Data Requirement
FLASH: Buoyancy-Driven Turbulent Nuclear Burning	300 TB
Reactor Core Hydrodynamics	5 TB
Computational Protein Structure	1 TB
Kinetics and Thermodynamics of Metal and Complex Hydride Nanoparticles	100 TB
Climate Science	345 TB
Parkinson's Disease	50 TB
Lattice QCD	44 TB

[Source: 2008 report, S. Klasky]

Compute vs. I/O trends



Parallel I/O

What?

- Every process reads and writes files in parallel
- Simultaneous access to storage (at least the illusion of it)

Why?

- Input/output data is of the order of TBs!
- Disk access rates are of the order of GB/s
- Speed up data availability in the process' memory

Sequential Writes

Problem?

```
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#define BUFSIZE 10000000

int main(int argc, char *argv[]) {

    int i, myrank, buf[BUFSIZE];
    char filename[128];
    FILE *myfile;

    strcpy(filename, "testfile");
    myfile = fopen(filename, "w");

    for (i=0; i<BUFSIZE; i++) {
        buf[i] = myrank + i;
        fprintf(myfile, "%d\n", buf[i]);
    }

    fclose(myfile);

    return 0;
}
```

Write to the Same File

File pointer

Solution?

Uncoordinated writes

```
#include "mpi.h"
#include <stdio.h>
#include <string.h>
#define BUFSIZE 10000

int main(int argc, char *argv[]) {

    int i, myrank, buf[BUFSIZE];
    char filename[128];
    FILE *myfile;

    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &myrank);

    strcpy(filename, "testfile");
    myfile = fopen(filename, "w");

    for (i=0; i<BUFSIZE; i++) {
        buf[i] = myrank + i;
        fprintf(myfile, "%d\n", buf[i]);
    }
    fclose(myfile);

    MPI_Finalize();
    return 0;
}
```

Write to Different Files

Independent writes



```
#include "mpi.h"
#include <stdio.h>
#define BUFSIZE 1000

int main(int argc, char *argv[]) {

    int i, myrank, buf[BUFSIZE];
    char filename[128];
    FILE *myfile;

    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &myrank);

    sprintf(filename, "testfile.%d", myrank);
    myfile = fopen(filename, "w");

    for (i=0; i<BUFSIZE; i++) {
        buf[i] = myrank + i;
        fprintf(myfile, "%d ", buf[i]);
    }
    fprintf(myfile, "\n");
    fclose(myfile);

    MPI_Finalize();
    return 0;
}
```

Timing (Different data sizes for $P=1$ and $P=4$)

0.0225s

1.911s

23.779s

0.0841s

4.251s

46.416s

Q: Problem with writing to different files per rank?

A: Think about 2D/3D domain decomposition

Simple Parallel I/O Code – Shared File

MPI_File fh

file_size_per_proc = FILESIZE / nprocs

MPI_File_open (MPI_COMM_WORLD, “/scratch/largefile”,
MPI_MODE_RDONLY, MPI_INFO_NULL, &fh)

Returns file handle

MPI_File_seek (fh, rank*file_size_per_proc, MPI_SEEK_SET)

MPI_File_read (fh, buffer, count, MPI_INT, status)

MPI_File_close (&fh)



Parallel Read using Explicit Offset

MPI_Offset offset = (MPI_Offset) rank*file_size_per_proc*sizeof(int)

MPI_File_open (MPI_COMM_WORLD, “/scratch/largefile”,
MPI_MODE_RDONLY, MPI_INFO_NULL, &fh)

MPI_File_read_at (fh, offset, buffer, count, MPI_INT, status)

MPI_File_close (&fh)



```

MPI_File fh;  // FILE

MPI_Init(&argc, &argv);
MPI_Comm_rank(MPI_COMM_WORLD, &myrank);
MPI_Comm_size(MPI_COMM_WORLD, &nprocs);

for (int i=0; i<BUFSIZE ; i++)
    buf[i]=i;

strcpy(filename, "testfileIO");

// File open, fh: individual file pointer
MPI_File_open (MPI_COMM_WORLD, filename, MPI_MODE_CREATE | MPI_MODE_RDWR, MPI_INFO_NULL, &fh);

MPI_Offset fo = (MPI_Offset) myrank*BUFSIZE*sizeof(int);

// File write using explicit offset (independent I/O)
MPI_File_write_at (fh, fo, buf, BUFSIZE, MPI_INT, MPI_STATUS_IGNORE); //fwrite

// File read using explicit offset (independent I/O)
MPI_File_read_at (fh, fo, rbuf, BUFSIZE, MPI_INT, &status); //fread

MPI_File_close (&fh);          //fclose

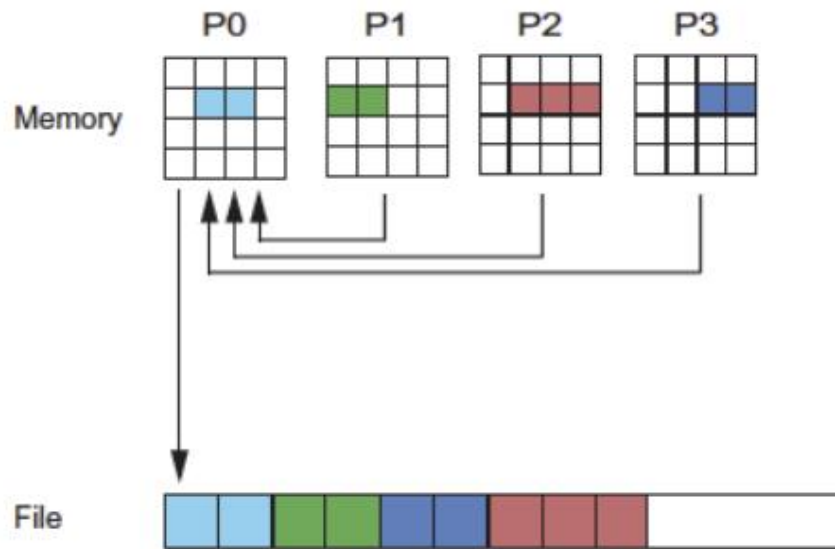
for (i=0; i<BUFSIZE ; i++)
    if (buf[i] != rbuf[i]) printf ("Mismatch [%d] %d %d\n", i, buf[i], rbuf[i]);

```

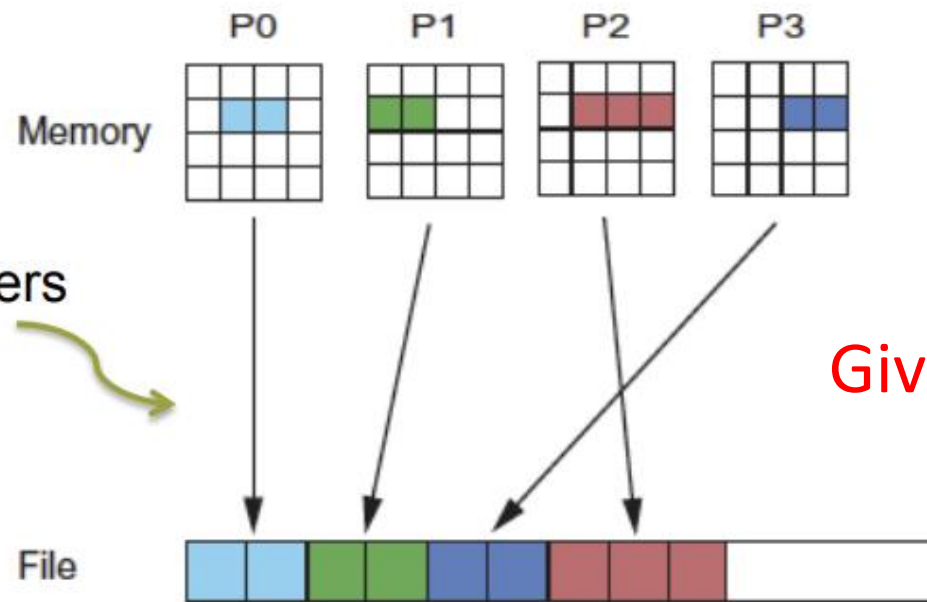
Quick Summary

- Multiple processes access a common file
- Multiple processes access the same file at the same time
- Multiple seeks issued at the same time
- Each process reads a **contiguous** chunk
- Individual file pointers





From: Single file, single writer



To: Single file, many writers

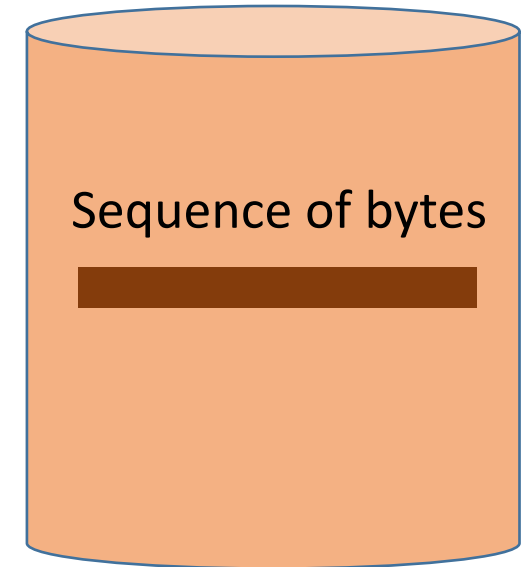
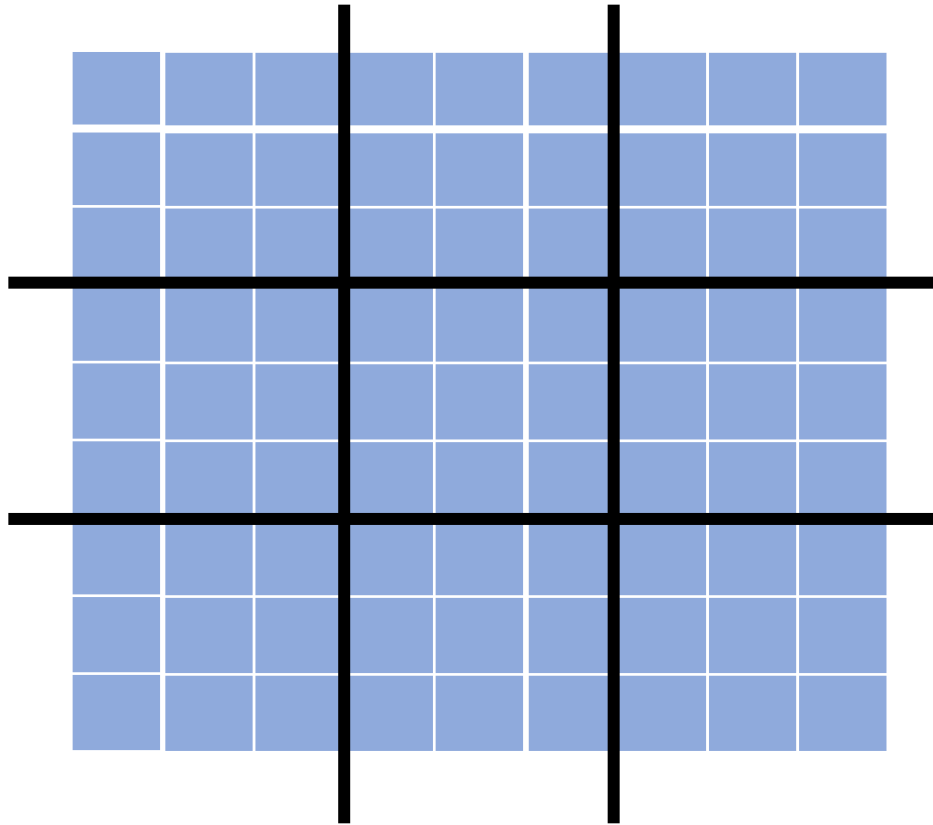
Give an example

Taken from "Getting Started on MPI-IO"
S-2490-40

COMPUTE |

Credit: CRAY presentation@IISc

Large Domain



Access pattern



Doubts

MPI_Bcast Timings (Lecture 15, Slide #8)

Multiple runs I: 0.022, 0.013, 0.012

Multiple runs III: 0.012, 0.012, 0.012

Multiple runs II: 0.019, 0.016, 0.016

Multiple runs IV: 0.025, 0.024, 0.025

```
CS633 $ mpirun -np 12 --hosts csews199:3,csews198:3,csews197:3,csews196:3 ./bcast_scale 100000
0.012307
CS633 $ mpirun -np 12 --hosts csews199,csews198,csews197,csews196 ./bcast_scale 100000
0.015753
CS633 $ mpirun -np 24 --hosts csews199:6,csews198:6,csews197:6,csews196:6 ./bcast_scale 100000
0.011843
CS633 $ mpirun -np 24 --hosts csews199,csews198,csews197,csews196 ./bcast_scale 100000
0.026107
```

Placement of processes and communication pattern together determines the communication performance.

Doubt

Lecture 15, Slide 8: I don't understand the timings **why is the first better than the second**, is it that the topology isn't linear? we could make an argument about how the second's communication would have a lot of congestion, but still if I had to **guess** I would've said the first test should take longer. Also, **why is the 3rd better than the 1st??**

```
CS633 $ mpirun -np 12 --hosts csews199:3,csews198:3,csews197:3,csews196:3 ./bcast_scale 100000
0.012307
CS633 $ mpirun -np 12 --hosts csews199,csews198,csews197,csews196 ./bcast_scale 100000
0.015753
```

More Runs

```
CS633 $ mpirun -np 16 --hosts csews199:4,csews198:4,csews197:4,csews196:4 ./bcast_scale 100000
0.012974
CS633 $ mpirun -np 16 --hosts csews199:4,csews198:4,csews197:4,csews196:4 ./bcast_scale 100000
0.013797
CS633 $ mpirun -np 16 --hosts csews199:4,csews198:4,csews197:4,csews196:4 ./bcast_scale 100000
0.013013
CS633 $ mpirun -np 16 --hosts csews199:4,csews198:4,csews197:4,csews196:4 ./bcast_scale 100000
0.013004
```

```
CS633 $ mpirun -np 32 --hosts csews199:8,csews198:8,csews197:8,csews196:8 ./bcast_scale 1000000
0.073156
CS633 $ mpirun -np 32 --hosts csews199:8,csews198:8,csews197:8,csews196:8 ./bcast_scale 1000000
0.072376
CS633 $ mpirun -np 32 --hosts csews199,csews198,csews197,csews196 ./bcast_scale 1000000
0.392194
CS633 $ mpirun -np 32 --hosts csews199,csews198,csews197,csews196 ./bcast_scale 1000000
1.179583
```

```

int message[arrSize];
int countArray[numtasks], displArray[numtasks];

int displ = 0;           // note that root process is 0 here

// this information is needed by the root
if (!rank)
    for (i = 0; i < numtasks; i++) {
        countArray[i] = arrSize*(i+1);           // depends on the counts, root may need to get it from the processes
        displArray[i] = displ;
        displ += countArray[i];
        printf ("%d %d %d\n", i, countArray[i], displArray[i]);
    }

if (!rank)
    printf ("\n");

// every process initializes their local array
srand(time(NULL));
for (i = 0; i < arrSize; i++) {
    message[i] = i; // (double)rand() / (double)RAND_MAX;
}

int recvMessage[displ];    // significant at the root process

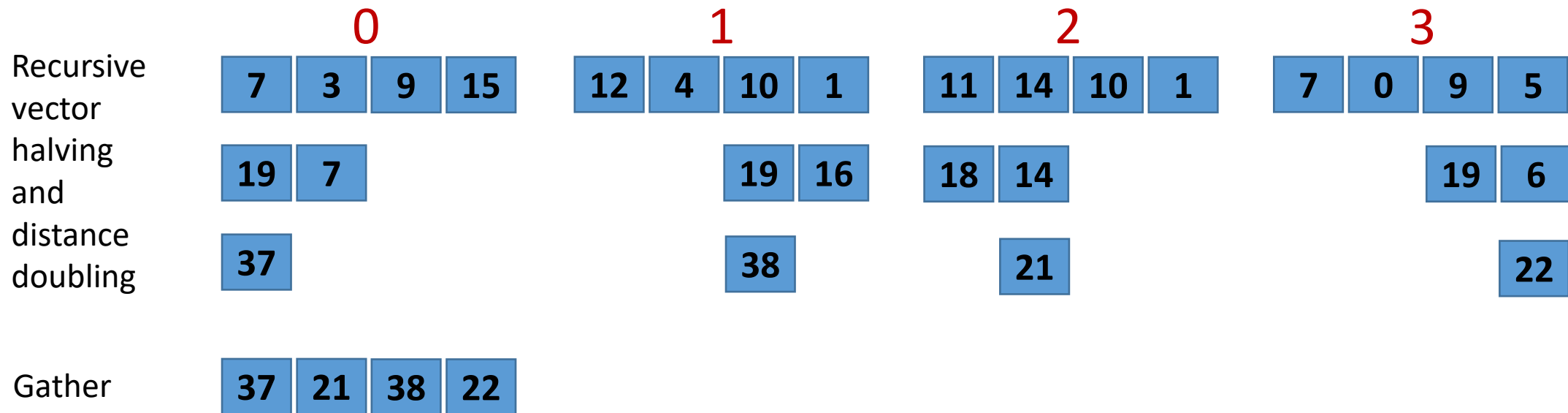
// receive different counts of elements from different processes
MPI_Gatherv (message, arrSize, MPI_INT, recvMessage, countArray, displArray, MPI_INT, 0, MPI_COMM_WORLD);

if (!rank)
    for (i = 0; i < displ; i++) {
        printf ("%d %d\n", i, recvMessage[i]);
    }

```

0	0
1	1
2	0
3	1
4	2
5	3

Reduce



In lecture 16, slide 40, Gather is not collecting elements in the same order as processes. But, the required answer for reduce is matching. Do we need to write Gather algorithm differently here? Same for algather in next slide. Could you please clarify this?

Another Example

node 0: recv B => AB

node 1: recv F => EF

node 2: recv D => CD

node 3: recv H => GH

node 4: send B

node 5: send F

node 6: send D

node 7: send H

node 0: recv CD => ABCD

node 1: recv GH => EFGH

node 2: send CD

node 3: send GH

node 0: recv EFGH => ABCDEFGH

Credit: cds.iisc.ac.in

Many-to-one Non-blocking P2P

while receiving the arr to recvarr[i], the position/order where we place the received message does not matter? Do we need to store in temp arr like in earlier lectures.

```
// send from all ranks to the last rank
start_time = MPI_Wtime ();
if (myrank < size-1)
{
    MPI_Send(arr, BUFSIZE, MPI_INT, size-1, 99, MPI_COMM_WORLD);
}
else
{
    int count, recvarr[size][BUFSIZE];
    for (int i=0; i<size-1; i++)
        MPI_Irecv(recvarr[i], BUFSIZE, MPI_INT, MPI_ANY_SOURCE, MPI_ANY_TAG, MPI_COMM_WORLD, &request[i]);
    MPI_Waitall (size-1, request, status);
}
time = MPI_Wtime () - start_time;

MPI_Reduce (&time, &max_time, 1, MPI_DOUBLE, MPI_MAX, size-1, MPI_COMM_WORLD);
if (myrank == size-1) printf ("Max time = %lf\n", max_time);
```

Lecture 16, Slide #19

```
CS633 $ time mpirun -np 12 --hosts csews98:6,csews99:6  
./gather 10000000
```

4.54 s

Assume a tree network
connecting the three hosts

```
CS633 $ time mpirun -np 12 --hosts  
csews98:4,csews99:4,csews100:4 ./gather 10000000
```

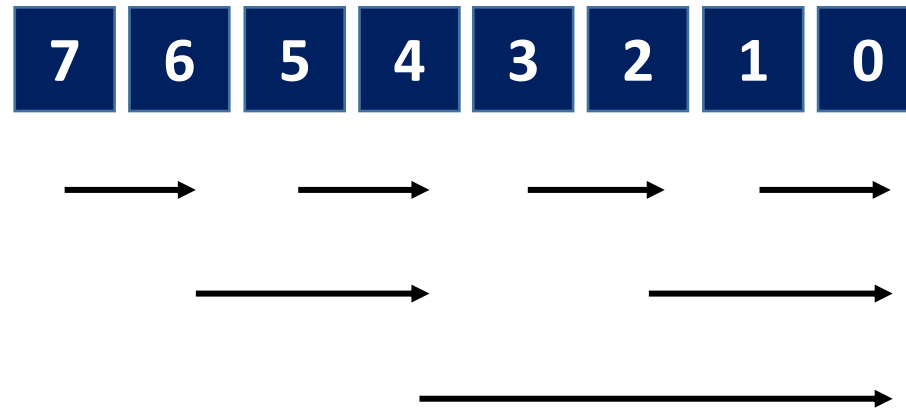
5.98 s

Doubt

Lecture no. 16, slide no. 19.

Assuming tree topology for nodes 98, 99, 100, first code execution has lesser time than the other (4.54s and 5.98s respectively). While when we assess how the communication goes during each round, it is found that both the executions have the same number of internode communications, that too without congestion, i.e. separately/independently. Then what is actually contributing to the difference between execution timings?

MPI_Gather Recursive Doubling Algorithm



Vector doubling
Distance doubling

```
CS633 $ time mpirun -np 12 --hosts csews98:6,csews99:6 ./gather 10000000
```

```
CS633 $ time mpirun -np 12 --hosts csews98:4,csews99:4,csews100:4 ./gather 10000000
```

```
CS633 $ time mpirun -np 16 --hosts csews102:8,csews101:8 ./gather 10000000

real    0m6.244s
user    0m0.031s
sys     0m0.011s
CS633 $ time mpirun -np 16 --hosts csews102:8,csews101:8 ./gather 10000000

real    0m6.018s
user    0m0.039s
sys     0m0.006s
CS633 $ time mpirun -np 16 --hosts csews12:8,csews11:8 ./gather 10000000

real    0m6.321s
user    0m0.033s
sys     0m0.005s
CS633 $ time mpirun -np 16 --hosts csews12:8,csews11:8 ./gather 10000000

real    0m6.021s
user    0m0.036s
sys     0m0.005s
```

```
CS633 $ time mpirun -np 16 --hosts csews12:6,csews11:6,csews21:4 ./gather 10000000
```

```
real    0m8.674s
user    0m0.051s
sys     0m0.011s
```

```
CS633 $ time mpirun -np 16 --hosts csews12:6,csews11:6,csews21:4 ./gather 10000000
```

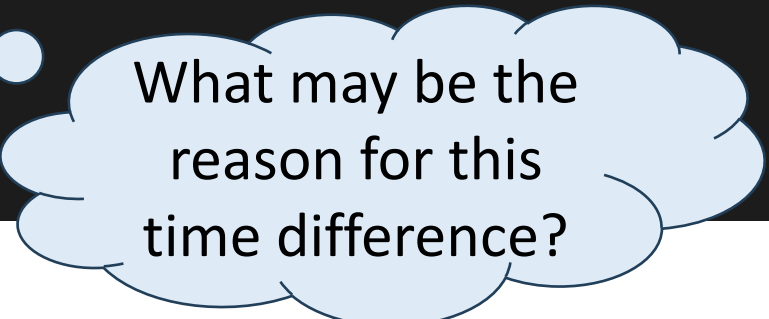
```
real    0m8.763s
user    0m0.058s
sys     0m0.014s
```

```
CS633 $ time mpirun -np 16 --hosts csews102:6,csews11:6,csews201:4 ./gather 10000000
```

```
real    0m17.254s
user    0m0.052s
sys     0m0.012s
```

```
CS633 $ time mpirun -np 16 --hosts csews102:6,csews11:6,csews201:4 ./gather 10000000
```

```
real    0m16.522s
user    0m0.062s
sys     0m0.011s
```



What may be the
reason for this
time difference?